

Datos Temporales

Sofia Chung

2025-10-17

Contents

Obtención de información	1
Análisis temporal	1

Obtención de información

Se obtuvo información del producto *Global Ocean Physics Reanalysis* de Copernicus (https://data.marine.copernicus.eu/product/GLOBAL_MULTIYEAR_PHY_001_030/download?dataset=cmems_mod_glo_phy_my_0.083deg_P1D-m_202311), con data en formato nc. de Temperatura Potencial Diaria a profundidad de 0.49m, y de periodo de un año correspondiente a 01/01/2021 a 31/12/2021, frente a las costas peruanas.

Análisis temporal

Abrimos el paquete descargado y exploramos sus variables para confirmar la información:

```
#install.packages("ncdf4")
library(ncdf4)
ncf <- nc_open("C:/Users/Sofia.Chung/Downloads/Personal/OHWe25/Talleres_intermedios/Evaluación_scchv/cm
#names(ncf$var)
#ncf$var$thetao

time <- ncf$dim$time$vals
head(time)
```

```
## [1] 1609459200 1609545600 1609632000 1609718400 1609804800 1609891200
```

Revisando el archivo nos percatamos que las fechas se encuentran en “segundos desde 1970/01/01” por lo que debemos convertirlo:

```
time <- as.POSIXct(time, tz = "UTC", format = "%Y-%m-%d",
                    origin = "1970-01-01 00:00:00")
head(time)
```

```
## [1] "2021-01-01 UTC" "2021-01-02 UTC" "2021-01-03 UTC" "2021-01-04 UTC"
## [5] "2021-01-05 UTC" "2021-01-06 UTC"
```

Verificamos que las fechas dadas se encuentran ahora en formato UTM para poder trabajarlas.

Con la función `read.cmems()` del paquete **satin**, importamos el archivo ara poder ver los datos diarios de temperatura potencial obtenidos:

```
# install.packages("devtools")
library(devtools)
```

```
## Cargando paquete requerido: usethis
```

```
#devtools::install_github("hvillalo/satin")
library(satin)
```

```
thetao <- read.cmems("C:/Users/Sofia.Chung/Downloads/Personal/OHWe25/Talleres_intermedios/Evaluación_sc
class(thetao)
```

```
## [1] "satin"
## attr(,"package")
## [1] "satin"
```

Después de confirmar que es un archivo satin, podemos explorar su contenido:

```
thetao
```

```
## Object of class satin
##
## Title: thetao
## Long name: Temperature
## Name: thetao
## Units: degrees_C
## Temporal range: daily
## Spatial resolution: 9.2 km
##
## Data dimensions:
## 185 187 181 1
##
## Data ranges:
##      lon      lat  thetao   period   depth
## min -86.83333 -18.50000 14.90680 2021-01-01 0.494025
## max -71.33333  -3.16667 30.23978 2021-06-30 0.494025
```

Verificamos que los valores corresponden a la temperatura potencia diaria en °C con 187 días de información pero que el periodo de fecha va desde 01/01/2021 al 30/06/2021, con un solo dato de profundidad de 0.49m y una resolución espacial de 9.2km.

Para ver los slots usamos "@":

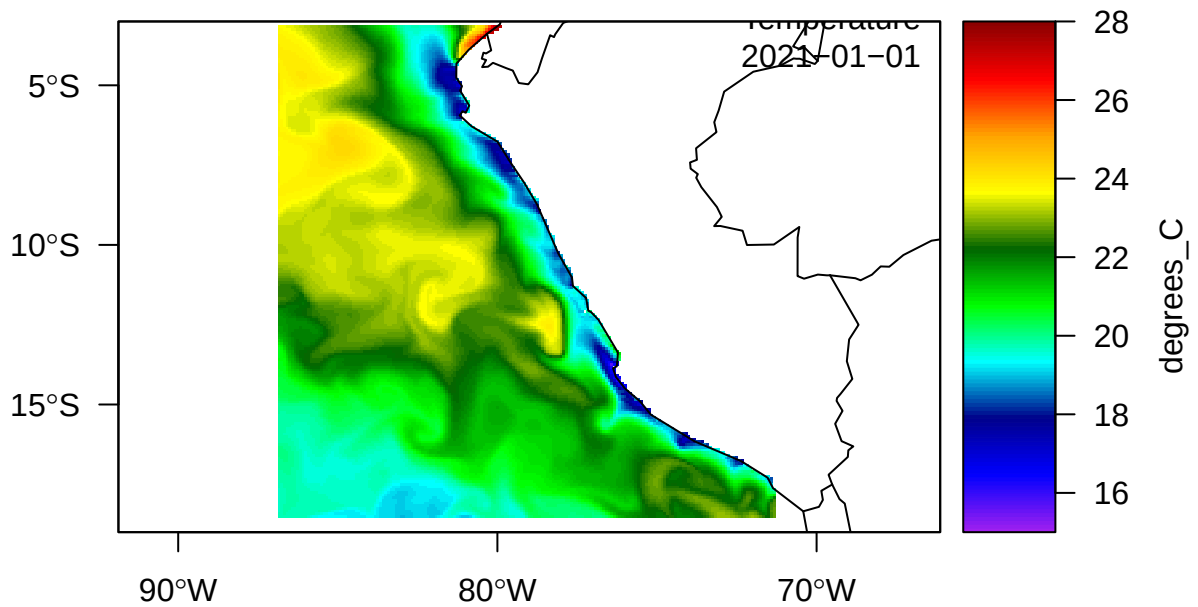
```
head(thetao@period$tmStart); tail(thetao@period$tmStart)
```

```
## [1] "2021-01-01 UTC" "2021-01-02 UTC" "2021-01-03 UTC" "2021-01-04 UTC"
## [5] "2021-01-05 UTC" "2021-01-06 UTC"
```

```
## [1] "2021-06-25 UTC" "2021-06-26 UTC" "2021-06-27 UTC" "2021-06-28 UTC"
## [5] "2021-06-29 UTC" "2021-06-30 UTC"
```

Podemos verificar los valores en una escala de la zona geográfica de nuestros datos:

```
plot(thetao)
```



Podemos tomar un pixel a los 12° de lat S y 77° de lon W (cerca a Lima, Perú)

```
pt <- data.frame(x = -77, y = -12.5)
sst <- extractPts(thetao, points = pt)
dim(sst)
```

```
## [1] 1 187
```

```
sst[, 1:10]
```

```
##   id   x    y d      lon  lat      p1      p2      p3      p4
## 1   1 -77 -12.5 0 -76.99999 -12.5 19.44649 19.65303 20.11228 20.2302
```

En `sst` están los valores de temperatura potencial para el punto frente a Lima que hemos seleccionado. Podemos usar también la siguiente función para “re-arreglar” los datos extraídos:

```
sst[ , 1:10]
```

```
##      id      x      y d      lon      lat      p1      p2      p3      p4
## 1      1 -77 -12.5 0 -76.99999 -12.5 19.44649 19.65303 20.11228 20.2302
```

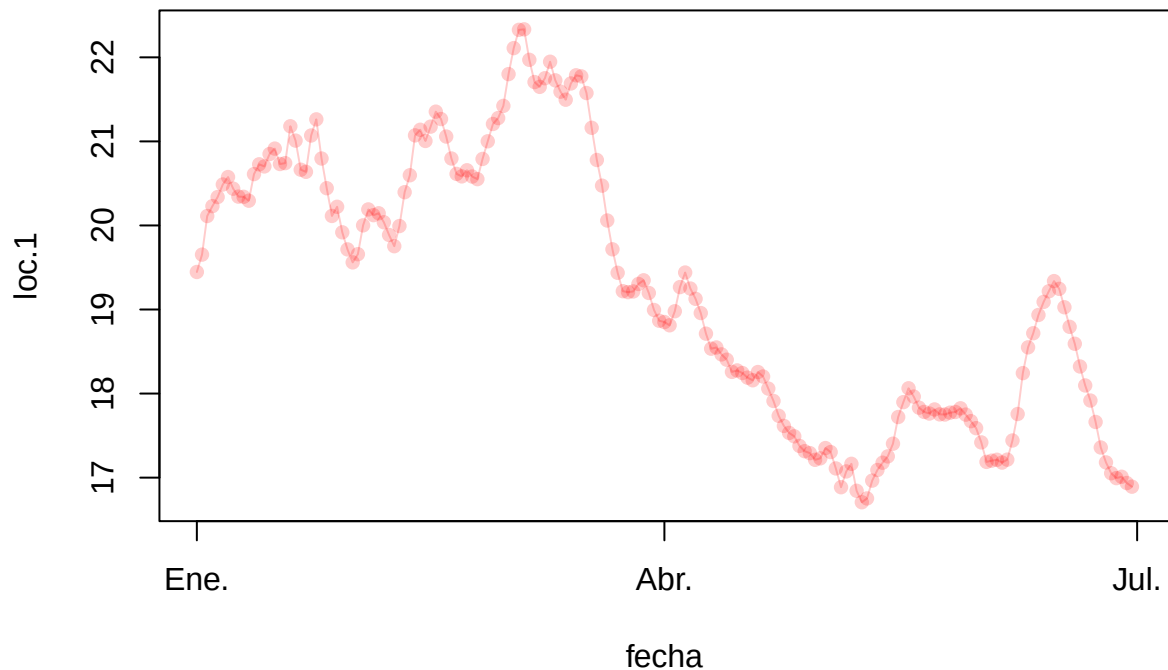
```
# función para darle formato a datos extraídos con extractPts()
formatXTP <- function(pts, plot.error = FALSE){
  error <- pts$d
  if (plot.error == TRUE){
    x11(); hist(error)
  }
  npts <- nrow(pts)
  ncols <- ncol(pts)
  att <- attributes(pts)
  ans <- data.frame(fecha = att$attribs$period$tmStart, t(pts[, 7:ncols]))
  rownames(ans) <- NULL
  names(ans)[2:(npts+1)] <- paste("loc", 1:npts, sep = ".")
  ans
}

tsm <- formatXTP(sst)
tsm <- tsm[1:181, ]
head(tsm)
```

```
##      fecha      loc.1
## 1 2021-01-01 19.44649
## 2 2021-01-02 19.65303
## 3 2021-01-03 20.11228
## 4 2021-01-04 20.23020
## 5 2021-01-05 20.33787
## 6 2021-01-06 20.48729
```

Con todo eso seteado, podemos graficar la serie de tiempo de nuestros datos usando la función `plot()`:

```
plot(tsm, type = "o", pch = 16, col = rgb(1, 0, 0, 0.2))
```



Y podemos suavizalo con la función `cma()` del paquete **smooth**. Como tenemos data de medio año aproximadamente, usaremos el orden de 5 para ser lo más precisos posibles:

```
#install.packages("smooth")
library(smooth)
```

```
## Cargando paquete requerido: greybox
```

```
## Package "greybox", v2.0.6 loaded.
```

```
## This is package "smooth", v4.3.0
```

```
sm <- cma(tsm$loc.1, order = 5)
tsm$sm <- sm$fitted

plot(tsm$fecha, tsm$loc.1, type = "b", pch = 16, col = rgb(1, 0, 0, 0.2),
     xlab = "2021", ylab = "temperatura potencial del mar (°C)")
lines(tsm$fecha, tsm$sm, lwd = 2.5)
```

